Quarterly Results

This report summarizes regional performance for the current fiscal quarter across four operating regions. Each region tracks bookings in two consecutive quarters and reports the resulting growth rate. The figures below are drawn from the finance team's internal reconciliation process and have been rounded to one decimal place where needed.

Region	Q1	Q2	Growth
EU	10	12	20%
US	7	9	28.6%
APAC	5	6	20%
LATAM	3	4	33.3%

Overall bookings grew across every region this quarter, with the strongest relative gains coming from smaller regions off a lower base. Management expects the growth rate in EU and APAC to converge with US and LATAM once the new billing pipeline finishes rolling out.

Storage Engines

The following table compares three embeddable storage engines commonly used as the foundation for larger database systems.

System	Description	Latency (ms)
RocksDB	An embeddable persistent key-value store built for fast flash storage and heavy write loads	1.2
LevelDB	A simple ordered key-value library	2.5
WiredTiger	A storage engine with document-level concurrency and compression	1.8

Each engine trades off write amplification, read latency and compression differently, so the right choice depends heavily on the access pattern of the workload sitting on top of it.

Design Notes

Building a durable storage layer starts with deciding how writes reach stable media before a caller is told the write succeeded. Many systems group incoming writes into a sequential log so that a single fsync call can cover a batch of unrelated operations. This amortizes the fixed cost of a disk flush across many logical writes and keeps write latency predictable even under bursty load. The log itself is treated as the ground truth and every other data structure the engine keeps in memory or on disk exists only to make reads faster.

Once a write is safely recorded in the log, the engine can update its in-memory structures at its own pace. A common approach is to buffer recent writes in a sorted in-memory table and only merge them into the larger on-disk structure once that table grows past a size threshold. This buffering absorbs bursts of writes without touching the disk on every single operation, and it lets the engine batch the eventual merge into a more efficient sequential write rather than many small random ones scattered across the file.

Reads are the harder problem, because a single key might have been written recently, written long ago or never written at all. A read path typically has to check the in-memory buffer first, then walk through progressively older on-disk structures until it either finds the key or exhausts every possible location. To avoid wasted work on structures that could not possibly contain the key, many engines keep a small probabilistic filter next to each on-disk structure that can quickly say whether a key is definitely absent, which turns most misses into a single cheap check rather than a full disk read.

Background compaction is what keeps this whole arrangement from degrading over time. Without it, the number of on-disk structures a read has to check would grow without bound and both read latency and space usage would climb accordingly. A compaction process periodically picks a handful of overlapping structures, merges them into fewer, larger ones and discards any entries that have since been overwritten or deleted. Tuning how aggressively compaction runs is one of the central trade-offs in this kind of design. Compacting often keeps reads fast and space usage low, but it consumes disk bandwidth and CPU that would otherwise go toward serving incoming traffic, so most production deployments end up choosing a compaction schedule that fits their own ratio of reads to writes rather than adopting a single universal default.

Latency Breakdown

The table below breaks down latency percentiles by workload type, followed by a small table of the configuration settings used to produce these measurements.

Workload	Latency (us)		Ops
	p50	p99	
read	120	450	1000
write	300	900	800
scan	800	2100	50

These numbers were collected on a single node with the storage engine configured as shown below, and no other workload running concurrently.

Setting	Value
Block size	4 KiB
Cache	512 MiB
Compression	LZ4

Table 1: Per-node throughput.

Node	Requests	Errors
node01	107	47
node02	114	84
node03	121	31
node04	128	68
node05	135	15
node06	142	52
node07	149	89
node08	156	36
node09	163	73
node10	170	20
node11	177	57
node12	184	94
node13	191	41
node14	198	78
node15	205	25
node16	212	62
node17	219	99
node18	226	46
node19	233	83
node20	240	30
node21	247	67
node22	254	14
node23	261	51
node24	268	88
node25	275	35
node26	282	72
node27	289	19
node28	296	56
node29	303	93
node30	310	40
node31	317	77
node32	324	24
node33	331	61
node34	338	98
node35	345	45
node36	352	82
node37	359	29
node38	366	66
node39	373	13
node40	380	50
node41	387	87
node42	394	34
node43	401	71
node44	408	18

Node	Requests	Errors
node45	415	55
node46	422	92
node47	429	39
node48	436	76
node49	443	23
node50	450	60
node51	457	97
node52	464	44
node53	471	81
node54	478	28
node55	485	65
node56	492	12
node57	499	49
node58	506	86
node59	513	33
node60	520	70
node61	527	17
node62	534	54
node63	541	91
node64	548	38
node65	555	75
node66	562	22
node67	569	59
node68	576	96
node69	583	43
node70	590	80

Table 2: Throughput after the upgrade.

Node	Requests	Errors
node01	220	3
node02	230	1
node03	240	0